



Detecção e Classificação de Golpes em Boxe com Visão Computacional

João Pedro Tamburo Faraoni, Leonardo de Sousa Marques,
Pedro Henrique Gimenez, Tom Pereira Hunt

Departamento de Informática e Estatística
Universidade Federal de Santa Catarina

{joao.faraoni, leonardo.sm, pedro.gimenez, tom.hunt}@grad.ufsc.br

1. Introdução

As artes marciais acompanham a humanidade desde a pré-história e, apesar de suas diferentes modalidades e estilos, permanecem amplamente praticadas tanto como atividade física quanto como forma de entretenimento. Nesse contexto, o **boxe** se destaca como um esporte de combate caracterizado pela combinação de técnicas ofensivas e defensivas [1] executadas em um ambiente altamente dinâmico.

Durante uma luta, os atletas realizam diferentes tipos de golpes, como **jab**, **cross**, **hook** e **uppercut**, além de movimentos defensivos e deslocamentos constantes. O desempenho do lutador depende não apenas da execução técnica desses movimentos, mas também de sua capacidade perceptiva e cognitiva para interpretar as ações do adversário e responder adequadamente [1].

Diante disso, podemos apontar que uma dificuldade dos telespectadores desse tipo de esporte é identificar os golpes realizados pelos lutadores durante a partida, devido sua similiaridade e rapidez. Isso abre espaço para o desenvolvimento de aplicações com visão computacional.

Dessa forma, o presente trabalho busca apresentar o `boxe.ml`¹ v1.0.0, uma ferramenta *open source*, projetada com Python 3.12, que recebe via linha de comando (CLI) um vídeo de uma partida de boxe e realiza o traqueamento dos lutadores e atua na detecção e classificação dos golpes realizados com uma determinada confiança. O vídeo completo da [apresentação de slides](#) do trabalho e da ferramenta pode ser acessado [neste link](#).

2. Revisão Bibliográfica

Primeiramente, para o desenvolvimento do projeto, realizou-se uma pesquisa acerca das

¹Repositório online: <https://github.com/leonardosm14/boxe.ml>

²Jabbr.ai: <https://jabbr.ai/>

tecnologias existentes. Percebeu-se a existência de soluções muito complexas, como o [Jabbr.ai](#)², porém que não estão disponibilizadas em código aberto.

Além disso, foram encontradas soluções com *Machine Learning* (ML), porém que não satisfazem os requisitos de identificação dos seis golpes primordiais do boxe, que só funcionam para um único lutador e em tempo real, sem possibilidade de análise de lutas em vídeo.

Tendo esse contexto, buscaram-se *datasets* com dados já anotados, a fim de diminuir o esforço e dificuldade do tema. Com isso, foi encontrado o artigo³ “**BoxingVI: A Multi-Modal Benchmark for Boxing Action Recognition and Localization**” [2]. No trabalho citado, foi criado um *benchmark* a partir da anotação de dados de 20 vídeos, com um total de 6.915 cliques anotados.

Por fim, realizou-se um estudo acerca do trabalho “**AlphaPose: Whole-Body Regional Multi-Person Pose Estimation and Tracking in Real-Time**” [3]. Tal ferramenta foi utilizada pelo paper do *benchmark* BoxingVI [2] para detecção de objetos (pessoas). Entendeu-se importante compreender o formato dos dados de anotação e dos esqueletos criados, a fim de estudar a viabilidade dos dados para utilização do *dataset*.

3. Datasets

A fonte de dados principal deste trabalho foi os dados disponibilizados no repositório público do BoxingVI [4]. Conforme a Figura 1, pode-se observar a sequência dos 15 vídeos originais utilizados para a criação do conjunto de treino

do *benchmark*. Todavia, na base de dados disponibilizada, foi dado acesso a apenas 9 vídeos.

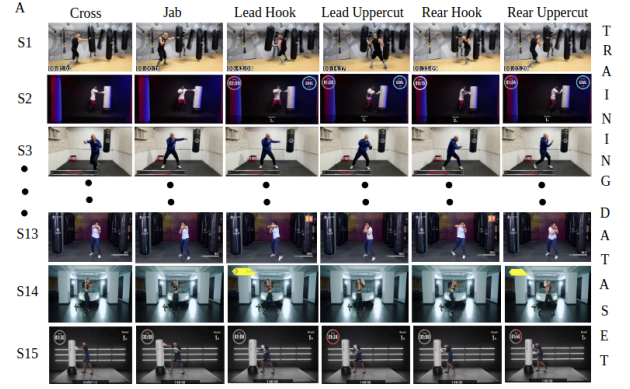


Figura 1: *Dataset* do BoxingVI [2].

No total, tivemos acesso a 5.463 dos cliques, um valor razoável quando comparado ao total do *benchmark*. A distribuição do total de golpes é vista na Tabela 1.

Classe	Quantidade
Cross	1373
Jab	1289
Lead Hook	1037
Lead Uppercut	689
Rear Hook	394
Rear Uppercut	681

Tabela 1: Distribuição de golpes.

O *dataset* possui três pastas principais: *annotation_files*, *skeleton_data* e *rgb_videos*. As duas primeiras são as mais importantes para a realização do trabalho. A pasta de anotação contém os nove arquivos no formato CSV, com as seguinte colunas: *start_frame* (frame inicial), *end_frame* (frame final) e *class* (classe), que descreve o golpe realizado naquela janela temporal. Vale ressaltar, também, que os golpes foram anotados sob os vídeos numa janela de 25 fps, o que impacta diretamente na medição dos dados. Na tabela Tabela 2, pode-se observar a formatação comentada, em relação ao vídeo 2 (V_2).

³Aponta-se, entretanto, que o trabalho é um *pre-print*. Ou seja, não foi oficialmente publicado em *journals*, revistas ou conferências.

id	start_frame	end_frame	class
1	6675	6688	Jab
2	6689	6697	Cross
3	6698	6710	Lead Hook
4	6722	6734	Jab
5	6735	6745	Cross
⋮	⋮	⋮	⋮

Tabela 2: CSV relativo à V_2 .

A pasta de dados dos esqueletos armazena os nove arquivos em formato NPY (binário padrão da biblioteca NumPy), de modo que cada arquivo contém as informações dos 17 principais pontos do corpo humano, conforme padronizado pelo Microsoft *Common Objects in Context* (COCO) [5], em determinado instante de tempo.

Os dados com os vídeos completos não foi utilizada na prática. Apenas foi realizado um *cross-check* para verificar a consistência das anotações e garantir a diversidade dos vídeos e lutadores.

4. Desenvolvimento

O `boxe.ml` foi projetado na linguagem de programação Python, versão 3.12. A estrutura do projeto segue a [Listagem 1](#) abaixo.

```

├─ dataset/
│   ├── clean_annotation_data/
│   ├── skeleton_data/
│   └─ rgb_videos/
├─ utils/
│   ├── stance_utils.py
│   └─ boxe_utils.py
├─ boxe.py
├─ stance.py
├─ tracking.py
└─ training.ipynb

```

Listagem 1: Estrutura do projeto.

4.1. Manipulação de Dados

Em primeiro plano, foi necessário realizar uma limpeza dos dados de anotação em CSV retirados de [4], visto que uma classe, em determinadas anotações, poderia estar escrita de formas di-

ferenças (*e.g.*, “Jab”, “jab”). Os dados foram então adicionados à pasta `clean_annotation_data`.

Como escolha de projeto, a fim de não atribuir à rede neural a função de diferenciar entre os seis modelos, optou-se por separá-los em apenas três classes: **Straight**, composta por Jab e Cross, **Uppercut**, composta por Lead Uppercut e Rear Uppercut, e **Hook**, composta por Rear Hook e Lead Hook. Tomou-se essa decisão após uma massiva testagem com as seis classes originais, e percebeu-se que, por conta do baixo volume de dados para algumas das classes, o modelo não conseguia atingir uma boa acurácia. Essa manipulação de dados é realizada diretamente no módulo de treinamento. Conforme a [Tabela 3](#), observa-se um melhor balanceamento dos dados quando comparados à [Tabela 1](#).

Classe	Conjunto	Quantidade
Straight	Jab $\cup$ Cross	2662
Uppercut	Lead Uppercut $\cup$ Rear Uppercut	1370
Hook	Lead Hook $\cup$ Rear Hook	1431

Tabela 3: União de dados.

Optou-se por utilizar os vídeos V_1 , V_2 , V_3 , V_4 , V_7 e V_8 para treinamento e validação, com um *split* aleatório de 65% e 35% dos dados para cada etapa, respectivamente – utilizou-se o método `train_test_split` da biblioteca `Scikit-learn`, com semente = 42, a fim de garantir reprodutibilidade dos resultados. Já para a etapa de teste do modelo, utilizou-se os vídeos V_5 , V_9 e V_{10} . Tanto os vídeos de treinamento/validação e teste foram separados de acordo com sua diversidade. Isso garante que nosso modelo de ML não “decore” os golpes baseando-se em um lutador, pose ou cenário em específico. Conforme a [Tabela 4](#),

observa-se a divisão gerada de golpes para cada uma das etapas mencionadas. No total, teve-se 2513 golpes de treinamento, 1354 golpes de validação e 899 golpes de teste.

Conjunto	Straight	Hook	Uppercut
Treino	1390	673	450
Validação	749	362	243
Teste	519	172	208

Tabela 4: Distribuição das amostras utilizadas nos conjuntos de treino, validação e teste.

Por fim, antes da criação e treinamento do modelo de ML, realizou-se a técnica de aumento de dados por espelhamento [6]. Como decidiu-se verificar se um golpe é *lead* ou *rear* somente na *pipeline*, percebeu-se a oportunidade de aumentar a quantidade de dados através do espelhamento. Portanto, como entrada do modelo, vamos ter o dobro de dados de treinamento (5026).

4.2. Modelo de *Machine Learning*

Depois de todo o tratamento de dados, foi possível criar o modelo de *Machine Learning* utilizado. Para este projeto, foi feito uso do Tensorflow v2.21.0, por conta da facilidade que a biblioteca oferece para criação de um modelo de ML em camadas no Python. Visto que se deseja prever movimentos em vídeos, optou-se por utilizar uma rede neural *Bidirectional Long Short-Term Memory* (Bi-LSTM), combinada com um mecanismo de atenção multi-cabeça. O modelo é composto pelas seguintes camadas:

1. Camada de Entrada (*Input Layer*).

Recebe sequências de esqueleto pré-processadas com shape (batch, 25, 102), onde 25 corresponde ao número de frames da janela temporal e 102 às features extraídas de 17 juntas corporais — coordenadas (x, y) , velocidade (v_x, v_y) e aceleração (a_x, a_y) para cada junta.

2. Bi-LSTM. É a parte mais importante do modelo, com 85.504 parâmetros. Um LSTM convencional lê a sequência de frames do início ao fim; a versão bidirecional executa dois LSTMs em paralelo — um no sentido direto (frame $i \rightarrow j$) e outro no sentido inverso (frame $j \rightarrow i$) — e concatena as saídas, resultando em 128 unidades. Isso é vantajoso porque um golpe possui contexto tanto no movimento de preparação quanto no de retração. Regularizações de *dropout* ($p = 0.30$) e *recurrent dropout* ($p = 0.20$) são aplicadas internamente.

3. Dropout ($p = 0.30$). Após o Bi-LSTM, 30% dos neurônios são desligados aleatoriamente a cada *batch* durante o treino. Trata-se de uma técnica de regularização que força o modelo a não depender de nenhum neurônio específico, reduzindo o overfitting.

4. Multi-Head Attention. Com 2 cabeças e dimensão de chave 32, esse mecanismo, inspirado na arquitetura Transformer, computa relações de atenção entre todos os frames simultaneamente, identificando quais instantes temporais são mais relevantes para a classificação do golpe. A operação é definida por

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) \cdot V$$

onde Q , K e V são projeções lineares da mesma entrada (*self-attention*).

5. Conexão Residual ($\text{Add} \oplus$). A saída do Dropout é somada à saída da camada de atenção. Essa *skip connection* garante que o gradiente flua diretamente pelo caminho residual durante o *back propagation*, mitigando o problema de gradiente que desaparece em redes mais profundas.

6. Normalização de Camada. Normaliza cada amostra individualmente ao longo da dimensão de features, estabilizando o treinamento

após a operação de adição residual. Possui 256 parâmetros treináveis.

7. Pooling Global por Média. Colapsa o eixo temporal calculando a média ao longo dos 25 frames, transformando o tensor de shape $(batch, 25, 128)$ em $(batch, 128)$. Essa agregação produz uma representação compacta de toda a sequência antes da classificação final.

8. Camada Densa com ReLU. Camada totalmente conectada com 64 unidades e ativação ReLU, responsável por aprender combinações não-lineares das *features* agregadas. Regularização L_2 ($\lambda = 5 \times 10^{-4}$) é aplicada aos pesos para penalizar valores muito grandes.

9. Dropout ($p = 0.30$). Segunda camada de *dropout*, aplicada após a densa, mantendo a regularização próxima à saída do modelo.

10. Camada de Saída (Softmax). Camada densa com 3 unidades e ativação *softmax*, que converte os logits em probabilidades para as três classes de golpe: Straight, Uppercut e Hook. A classe predita é aquela com maior probabilidade.

No Algoritmo 2 abaixo, pode-se observar o pseudo-código que descreve a criação do modelo, seguindo as camadas supracitadas, em que *shape* corresponde à dimensão de cada amostra de entrada, e N é o número de classes (3).

```

1 function BuildModel(shape, N):
2   i ← Input(shape)
3   x ← BiLSTM(64, 0.3, 0.2)(inp)
4   x ← Dropout(0.3)(x)
5   att ← MultiHeadAttention(2, 32)(x, x)
6   x ← LayerNorm(x + att)
7   x ← GlobalAveragePooling(x)
8   x ← Dense(64, ReLU,  $5 \times 10^{-4}$ )(x)
9   x ← Dropout(0.3)(x)
10  return Model(i, Dense(N, softmax)(x))

```

Algoritmo 2: Modelo com rede neural Bi-LSTM

O modelo proposto com camadas Bi-LSTM e atenção multi-cabeça resulta em uma arquitetura compacta de apenas 127.299 parâmetros treináveis, conforme descrito na Tabela 5.

Parâmetros	Quantidade	Tamanho
Treináveis	127.299	497,26 KB
Não-treináveis	0	0,00 B
Total	127.299	497,26 KB

Tabela 5: Parâmetros do modelo proposto.

4.3. Balanceamento de Classes

Para lidar com o desbalanceamento entre as classes, foram atribuídos pesos inversamente proporcionais à frequência de cada classe durante o treinamento, calculados via

$$w_c = \frac{N}{C \cdot n_c},$$

onde N é o total de amostras, C o número de classes e n_c o número de amostras da classe c . Ainda assim, mesmo após a duplicação das amostras por espelhamento horizontal, a disparidade entre as classes permaneceu acentuada — especialmente para Hook e Uppercut, que apresentaram frequência significativamente menor que Straight. Por isso, os pesos dessas duas classes foram amplificados por um fator adicional de 2.5, escolhido empiricamente, resultando nos valores apresentados na Tabela 6.

Classe	Peso
Hook	3,11
Straight	0,60
Uppercut	4,65

Tabela 6: Pesos por classe para o treinamento.

4.4. Callbacks

O treinamento foi controlado por três *callbacks* complementares. Da biblioteca do TensorFlow, foi utilizado o método *EarlyStopping*, que interrompe o treinamento caso a acurácia de

validação não melhora por 15 épocas consecutivas, restaurando automaticamente os pesos da melhor época observada. Também utilizou-se o `ReduceLRonPlateau`, com o objetivo de reduzir a taxa de aprendizado pela metade sempre que a perda de validação estagna por 5 épocas, com valor mínimo de 10^{-6} , permitindo refinamentos mais finos na convergência. Por fim, o `ModelCheckpoint` salva em disco apenas o modelo, em formato `keras`, com melhor acurácia de validação ao longo do treinamento, garantindo que o modelo final avaliado seja o ótimo encontrado e não necessariamente o da última época.

5. Resultados do Treinamento

O treinamento do modelo descrito na seção anterior foi realizado utilizando o método `fit` do TensorFlow. Foi definido um máximo de 100 épocas, considerando a utilização dos *callbacks* apresentados anteriormente, e um tamanho de *batch* de 32 amostras. O treinamento foi interrompido antecipadamente na época 48 pelo mecanismo de *Early Stopping*, após 15 épocas consecutivas sem melhoria na acurácia de validação (`val_accuracy`), totalizando aproximadamente 32 minutos de execução. A [Tabela 7](#) apresenta, para a última época de treinamento e para o melhor modelo restaurado, as métricas de acurácia e perdas (`loss`) dos conjuntos de treino e validação. A acurácia representa a proporção de amostras classificadas corretamente, enquanto a `loss` corresponde ao valor da função de custo de entropia cruzada categórica, sendo desejáveis valores menores. As métricas obtidas no conjunto de validação são as mais relevantes para avaliar a capacidade de generalização do modelo, pois refletem seu desempenho em dados não utilizados durante o treinamento. O melhor modelo foi obtido na época 33, alcançando 85,16% de

acurácia de validação e perda de 44,70%. Embora a última época tenha apresentado acurácia de treino ligeiramente superior (81,38%), os pesos restaurados da época 33 foram mantidos por apresentarem o melhor desempenho no conjunto de validação, indicando boa capacidade de generalização e ausência de sinais significativos de *overfitting*.

Época	Acc. Treino	Loss Treino	Acc. Val.	Loss Val.
1	0,2809	2,0910	0,2792	1,4372
2	0,3211	1,9537	0,3900	1,2142
⋮	⋮	⋮	⋮	⋮
33	0,7939	0,8769	0,8516	0,4470
⋮	⋮	⋮	⋮	⋮
47	0,8086	0,8088	0,8442	0,4553
48	0,8138	0,8162	0,8353	0,4687

Tabela 7: Evolução das métricas de treino e validação ao longo do treinamento.

Após a validação do modelo com os vídeos V_5 , V_9 e V_{10} , os resultados se mostraram consistentes, atingindo uma acurácia de 83% sobre 899 amostras. A [Tabela 8](#) detalha as métricas por classe: precisão, revocação e F1-score. A **precisão** mede a proporção de predições positivas que estão corretas; a **revocação** mede a proporção de amostras reais de cada classe que foram corretamente identificadas; e o **F1-score** é a média harmônica entre ambas, sendo a métrica mais equilibrada quando há desbalanceamento entre classes. A média macro, que trata todas as classes com peso igual, atingiu 78%, enquanto a média ponderada, que considera o número de amostras de cada classe, acompanhou a acurácia geral de 83%, refletindo o bom desempenho na classe majoritária (Straight). O menor F1-score foi observado na classe Hook (0,68), o que era esperado dado o menor volume de amostras e

a maior similaridade visual desse golpe com as demais categorias.

Classe	Precisão	Revocação	F1-score	Suporte
Hook	0,68	0,69	0,68	172
Straight	0,91	0,91	0,91	519
Uppercut	0,76	0,76	0,76	208
Média macro	0,78	0,78	0,78	899
Média ponderada	0,83	0,83	0,83	899

Tabela 8: Métricas de classificação por classe no conjunto de teste.

A Figura 3 apresenta a matriz de confusão obtida no conjunto de teste. A diagonal principal concentra a grande maioria das predições corretas: 118 de 172 Hooks (69%), 471 de 519 Straights (91%) e 158 de 208 Uppercuts (76%). Os erros mais frequentes ocorrem na classe Hook, que confunde 25 amostras com Straight e 29 com Uppercut – o que é esperado, dado que o Hook é biomecanicamente mais próximo dos demais golpes quando observado apenas pelo esqueleto 2D. Straight e Uppercut apresentam confusão mútua mais baixa (21 amostras em cada direção), refletindo a maior distinção visual entre esses movimentos.

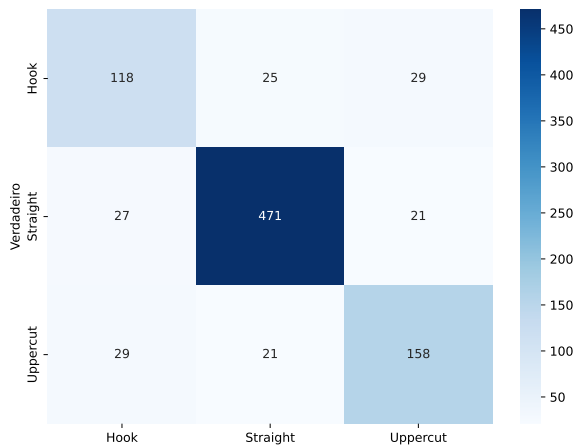


Figura 3: Matriz de confusão – V_5 , V_9 e V_{10} .

6. Pipeline

Com o modelo pronto e testado, iniciou-se a parte final deste trabalho, que foi desenvolver uma *pipeline* que realiza os passos descritos abaixo, a partir da execução do arquivo `boxe.py`:

1. Receber vídeo MP4 de entrada;
2. Converter vídeo para 25 fps, com FFmpeg⁴;
3. Carregar vídeo com OpenCV⁵;
4. Extrair esqueletos com YOLOv8m-Pose (COCO 17, coordenadas x, y) + *tracking*;
5. Carregar modelo Bi-LSTM;
6. Aplicar modelo sobre janelas do esqueleto;
7. Inferir *lead* ou *rear* pela geometria do golpe (punho que golpeou + perna da frente);
8. Escrever predições sobre os frames;
9. Gerar o vídeo anotado.

6.1. Tracking de Dois Lutadores

A presença de dois lutadores no vídeo introduz um desafio que vai além da simples detecção de poses: é necessário manter a identidade de cada boxeador ao longo do tempo, mesmo quando eles se cruzam ou saem momentaneamente do enquadramento. Para isso, utilizou-se o YOLOv8m-Pose em conjunto com o algoritmo de rastreamento ByteTrack, integrado nativamente ao framework Ultralytics [7]. A cada frame, o modelo detecta todas as pessoas presentes e, para cada detecção, retorna 17 *keypoints* no padrão COCO (coordenadas x, y) normalizadas pela resolução do vídeo e uma confiança por junta, além de uma caixa delimitadora em pixels e um *track_id* persistente atribuído pelo ByteTrack [8]. Detecções com confiança média inferior a 50% são descartadas. A atribuição de identidade a cada boxeador, denominados **Boxer 1** e **Boxer 2**, é realizada em dois estágios a partir das detec-

⁴Repositório do FFmpeg: <https://github.com/ffmpeg/ffmpeg>

⁵Repositório do OpenCV: <https://github.com/opencv/opencv>

ções brutas. No primeiro, cada detecção com um `track_id` previamente associado a um dos dois `slots` é diretamente atribuída a esse `slot`, independentemente de sua posição na tela, garantindo que a identidade de cada lutador se mantenha estável mesmo após cruzamentos de lado. No segundo estágio, detecções sem `track_id` ou com ID desconhecido (situações comuns no frame inicial ou após oclusões prolongadas) são atribuídas pela posição horizontal: a detecção mais à esquerda é associada ao Boxer 1 e a mais à direita ao Boxer 2. Quando um novo ID é atribuído por posição, o `slot` o registra para os frames seguintes, absorvendo naturalmente fragmentações do rastreador. Como o traqueamento é esparsos por natureza, a matriz de esqueletos de cada boxeador contém lacunas, frames em que o lutador não foi detectado, representadas por linhas de zeros. Uma lacuna implica que o punho “teleporta” para a origem e retorna ao frame seguinte, gerando um falso pico de velocidade e um evento de golpe sem contexto. Para evitar isso, frames sem detecção são preenchidos com a última pose conhecida daquele boxeador, mantendo a continuidade do sinal. Adicionalmente, o parâmetro `MIN_PRESENCE_RATIO` estabelece um limiar mínimo de presença: `slots` detectados em menos de 10% dos frames do clipe, gerados por reflexos, sombras ou outras interferências visuais momentâneas são descartados da renderização e da rotulação, evitando a criação de um segundo boxeador fantasma em clipes com apenas um atleta. Na Figura 4, observamos a saída final da *pipeline* com o traqueamento completo dos lutadores e inferência do golpe, discutido na Seção 6.2. Como referência, utilizou-se o vídeo `videos/fight.mp4`, presente no repositório.

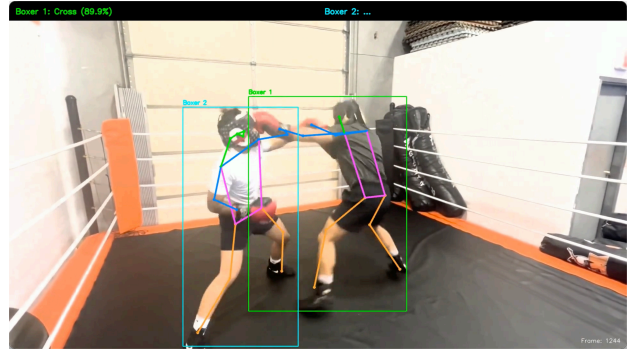


Figura 4: Saída da *pipeline* com dois lutadores: identidade persistente (Boxer 1/Boxer 2), esqueletos COCO-17 e classe final do golpe com a mão inferida (*lead/rear*) por boxeador.

6.2. Inferência do Golpe

O modelo de classificação descrito anteriormente prevê apenas o **tipo** do golpe (Straight, Hook ou Uppercut). A distinção entre a mão da frente (*lead*) e a mão de trás (*rear*) – que separa, por exemplo, o Jab do Cross – é realizada na própria *pipeline*, por geometria pura sobre os *keypoints*, sem um segundo modelo treinado. Essa separação é consequência direta das escolhas de projeto: a aumento por espelhamento horizontal só dobra os dados de treinamento porque a responsabilidade de distinguir a mão do golpe foi removida do modelo.

A primeira abordagem testada foi inferir a guarda (*stance*) de forma estática, comparando a posição dos tornozelos e a orientação do tronco para decidir qual pé está à frente. Essa estratégia se mostrou equivalente a ruído neste cenário, por duas razões verificadas empiricamente: o YOLOv8m-Pose sempre retorna as 17 juntas, “alucinando” a posição do lado ocluído do corpo, e a câmera lateral colapsa a profundidade frente/trás na projeção 2D. Agregar a guarda em um único valor por clipe também degrada o resultado (de 0,76 para 0,61 na validação cruzada entre vídeos), pois a orientação do lutador em relação à câmera varia ao longo do clipe.

A decisão adotada é feita **por golpe**, usando o sinal dinâmico do próprio soco: (i) a mão que golpeou é o punho com maior deslocamento líquido na janela do evento (pico de $\|p_t - p_0\|$), com desempate pelo maior alcance punho-ombro quando os deslocamentos diferem menos de 10%; (ii) o sinal do deslocamento horizontal desse punho define a direção “frente” local do golpe; (iii) a perna (média de joelho e tornozelo) mais avançada nessa direção indica o pé da frente. Se a mão que golpeou está do mesmo lado do pé da frente, o golpe é *lead*; caso contrário, *rear*. A classe final é obtida pelo mapeamento direto: Straight vira Jab (*lead*) ou Cross (*rear*), e Hook e Uppercut recebem o prefixo Lead ou Rear.

Um detalhe de implementação relevante: a janela usada nessa decisão emprega o esqueleto **sem** a suavização temporal aplicada no restante da *pipeline*. Um soco rápido é um movimento de ida e volta em poucos frames, e a média móvel de 5 frames achata justamente o deslocamento do punho que golpeou – com janelas suavizadas, a acurácia *lead/rear* no vídeo de validação próprio cai de 0,89 para 0,28.

Avaliada isoladamente sobre as janelas anotadas do BoxingVI, a geometria atinge 0,85 de acurácia *lead/rear* no conjunto de treino, 0,77 em validação cruzada entre vídeos e 0,74 no conjunto de teste. Em um vídeo gravado pelo próprio grupo e anotado manualmente (18 golpes, câmera traseira em ângulo de três quartos), a decisão por segmento anotado acerta 16 de 18 golpes (0,89), indicando que a abordagem geométrica generaliza para pontos de vista diferentes dos do dataset. A avaliação completa pode ser reproduzida pelo script `eval_leadrear.py` do [repositório](#).

6.3. Setup e Utilização via CLI

O primeiro passo é clonar o [repositório](#) do `boxe.ml`. Depois, é necessário instalar as dependências (TensorFlow, Ultralytics, ...) para conseguir executar a *pipeline*. Todas as dependências requisitadas foram listadas no arquivo `requirements.txt`. Para instalar, siga a [Listagem 2](#). Sugere-se que o usuário crie um ambiente virtual do Python para realizar essa instalação, porém os dois primeiros comandos são opcionais.

```
python3 -m venv venv \
source venv/bin/activate \
pip3 install -r requirements.txt
```

Listagem 2: Instalação dos requisitos (Linux).

Com isso, já é possível utilizar o código para detecção e classificação de golpes em vídeos selecionados. Basta executar o comando da [Listagem 3](#). A *flag* `--video` (ou `-v`) indica o caminho para o vídeo em MP4 desejado, enquanto `--output` (ou `-o`) indica a pasta em que o vídeo final analisado será armazenado. Existem *flags* adicionais, como `--model` (ou `-m`), para que o usuário possa carregar seu próprio modelo (no formato `keras`), caso desejado – por padrão, o módulo carrega o `modelo_boxe.keras` treinado para este projeto, também presente no repositório. E a *flag* `--clear-cache`, para que os esqueletos de um vídeo já processado sejam removidos da pasta e a *pipeline* execute do zero.

```
python3 boxe.py \
--video path/to/your/video.mp4 \
--output output/
```

Listagem 3: Execução da *pipeline*.

A documentação completa de como executar os *scripts* também está descrita no arquivo `README`, no [repositório](#).

7. Considerações Finais

O trabalho apresentando cumpriu com o objetivo de desenvolver uma *pipeline* totalmente automatizada para resolver o problema da detecção e classificação de golpes em partidas de boxe.

O uso de ferramentas tecnológicas já consolidadas, como TensorFlow, YOLO e outras, possibilitou o alcance de ótimos resultados para um problema muito complexo: realizar predição com modelos de *machine learning* em sequências temporais (vídeos), considerando características como velocidade, aceleração e posição dos lutadores.

Além disso, a solução de traqueamento possibilita que essa solução seja utilizada em ecossistemas reais de lutas de boxe, visto a persistência dos lutadores durante a partida. Como trabalho futuro, busca-se a implementação de um sistema de pontuação para cada boxeador, considerando os golpes inferidos.

Por fim, a fim de melhorar o modelo, sugere-se uma nova anotação de dados, com maior diversidade de lutadores, iluminação e ambientes, a fim de aumentar a acurácia final ainda mais.

Como última observação, apontamos que este trabalho fez uso de assistentes de Inteligência Artificial Generativa (IAGen) como ferramenta de apoio para pesquisa, *boilerplate*, depuração e revisão. Todavia, todas as decisões de projeto, a modelagem, os experimentos e a validação dos resultados foram feitos e conferidos pelos integrantes do grupo. Mais informações podem ser encontradas no arquivo `PROMPT.md` no [repositório](#).

Referências Bibliográficas

- [1] F. Loio Pinto *et al.*, «The efficiency and transversality of traditional boxing skills to several full-contact combat sports: a narrative review», *Archives of Budo Science of Martial Arts and Extreme Sports*, vol. 18, pp. 1–10, 2023.
- [2] R. Kumar *et al.*, «BoxingVI: A Multi-Modal Benchmark for Boxing Action Recognition and Localization», *arXiv preprint arXiv:2511.16524*, 2025, doi: [10.48550/arXiv.2511.16524](https://doi.org/10.48550/arXiv.2511.16524).
- [3] H.-S. Fang *et al.*, «AlphaPose: Whole-Body Regional Multi-Person Pose Estimation and Tracking in Real-Time», *arXiv preprint arXiv:2211.03375*, 2022, doi: [10.48550/arXiv.2211.03375](https://doi.org/10.48550/arXiv.2211.03375).
- [4] R. Kumar *et al.*, «BoxingVI: A Multi-Modal Benchmark for Boxing Action Recognition and Localization». [Em linha]. Disponível em: <https://github.com/Bikudebug/BoxingVI>
- [5] T.-Y. Lin *et al.*, «Microsoft COCO: Common Objects in Context», em *European Conference on Computer Vision (ECCV)*, 2014, pp. 740–755. doi: [10.1007/978-3-319-10602-1_48](https://doi.org/10.1007/978-3-319-10602-1_48).
- [6] C. Zhang *et al.*, «A Deep Learning Image Data Augmentation Method for Single Tumor Segmentation», *Frontiers in Oncology*, 2022, doi: [10.3389/fonc.2022.782988](https://doi.org/10.3389/fonc.2022.782988).
- [7] G. Jocher, A. Chaurasia, e J. Qiu, «Ultralytics YOLOv8». [Em linha]. Disponível em: <https://github.com/ultralytics/ultralytics>
- [8] Y. Zhang *et al.*, «ByteTrack: Multi-Object Tracking by Associating Every Detection Box», em *Proceedings of the 17th European Conference on Computer Vision (ECCV)*, Springer, 2022, pp. 1–21. doi: [10.1007/978-3-031-20047-2_1](https://doi.org/10.1007/978-3-031-20047-2_1).